计算机科学与技术学院 神经网络与深度学习课程实验报告

实验题目：文本分类		学号：202200130077
日期：2024.11.3	班级：22 级人工智能	姓名：薛润泽
Email：		

实验目的：

实现基于几种常见的深度学习方法（全连接网络、卷积神经网络 CNN、循环神经网络 RNN、Transformer）完成文本分类任务，并对实现的算法进行量化评测。通过实践，加深对文本分类任务的理解，掌握使用 PyTorch 进行深度学习模型构建和训练的方法。

实验软件和硬件环境：

Windows 和 Python 语言

实验原理和方法：

1、数据预处理：使用 torchtext 库下载 AG_NEWS 数据集，并进行分词和构建词汇表。

2、模型构建：分别构建基于全连接网络、CNN、RNN、Transformer 的文本分类模型。

3、模型训练：使用 DataLoader 读取数据集，构建训练循环，训练模型。

4、模型评估：构建评测函数，评估模型在测试集上的准确率等指标。

实验步骤：（不要求罗列完整源代码）

1、加载数据集

```
import torch
from torchtext.datasets import AG_NEWS
from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator
from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence

# 加载数据集
train_iter = AG_NEWS(split='train')
tokenizer = get_tokenizer('basic_english')
# 构建词汇表
def yield_tokens(data_iter):
    for _, text in data_iter:
        yield tokenizer(text)
vocab = build_vocab_from_iterator(yield_tokens(train_iter), specials=["<unk>"])
vocab.set_default_index(vocab["<unk>"])
```

2、数据处理

```
# 数据处理
def text_pipeline(text):
    return vocab(tokenizer(text))

def label_pipeline(label):
    return int(label) - 1

# DataLoader 和批量数据处理
def collate_batch(batch):
    label_list, text_list = [], []
    for (_label, _text) in batch:
         label_list.append(label_pipeline(_label))
         text_list.append(torch.tensor(text_pipeline(_text), dtype=torch.int64))
    return torch.tensor(label_list), pad_sequence(text_list, batch_first=True, padding_value=0)
train_iter, test_iter = AG_NEWS(split=('train', 'test'))
train_dataloader = DataLoader(list(train_iter), batch_size=8, shuffle=True,
collate_fn=collate_batch)
test_dataloader = DataLoader(list(test_iter), batch_size=8, shuffle=False,
collate_fn=collate_batch)

import torch.nn as nn
```

3、构建全连接文本分类模型

```
device = 'cpu'
class FullyConnectedModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_class):
        super(FullyConnectedModel, self).__init__()
        self.embedding = nn.EmbeddingBag(vocab_size, embed_dim, sparse=True)
        self.fc = nn.Linear(embed_dim, num_class)

    def forward(self, text):
        embedded = self.embedding(text)
        return self.fc(embedded)

num_class = len(set([label for (label, _) in train_iter]))
```

4、构建 CNN 文本分类模型

```
class CNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_class, kernel_sizes=[3, 4, 5],
num_channels=100):
        super(CNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.convs = nn.ModuleList([
```

```
            nn.Conv2d(1, num_channels, (k, embed_dim)) for k in kernel_sizes
        ])
        self.fc = nn.Linear(num_channels * len(kernel_sizes), num_class)

    def forward(self, text):
        embedded = self.embedding(text).unsqueeze(1)
        conv_outs = [torch.relu(conv(embedded)).squeeze(3) for conv in self.convs]
        pooled_outs = [torch.max(out, 2)[0] for out in conv_outs]
        cat_out = torch.cat(pooled_outs, 1)
        return self.fc(cat_out)
```

5、构建 RNN 文本分类模型

```
class RNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_class):
        super(RNNModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.rnn = nn.LSTM(embed_dim, hidden_dim, bidirectional=True, batch_first=True)
        self.fc = nn.Linear(hidden_dim * 2, num_class)

    def forward(self, text):
        embedded = self.embedding(text)
        rnn_out, _ = self.rnn(embedded)
        final_feature_map = rnn_out[:, -1, :]
        return self.fc(final_feature_map)
```

6、模型训练与评估

```
import torch.nn.functional as F
import random
def train(model, dataloader, optimizer, criterion, batch_size=128, num_classes=4):
    model.train()
    total_acc, total_count = 0, 0

    # 随机选择一批数据的索引
    indices = random.sample(range(len(dataloader.dataset)), batch_size)

    for idx in indices:
        label, text = dataloader.dataset[idx]

        if isinstance(text, str):
            text = text_pipeline(text)
            text = torch.tensor(text).unsqueeze(0)
        else:
            text = torch.tensor(text).unsqueeze(0)

        label = torch.tensor(label).unsqueeze(0)
```

```
        # 转移到设备
        text = text.to(device)
        label = label.to(device)

        optimizer.zero_grad()
        predicted_label = model(text)

        # 调试信息
        # print(f"Predicted: {predicted_label.argmax(1)}, Target: {label}")

        # 检查 label 是否超出范围
        if label.item() >= num_classes:
            # print(f"Invalid label value: {label.item()} (max class index: {num_classes - 1})")
            continue  # 跳过这次迭代，避免计算损失

        loss = criterion(predicted_label, label)
        loss.backward()
        optimizer.step()

        total_acc += (predicted_label.argmax(1) == label).sum().item()
        total_count += label.size(0)
    return total_acc / total_count if total_count > 0 else 0
def evaluate(model, dataloader):
    model.eval()
    total_acc, total_count = 0, 0
    with torch.no_grad():
        for idx, (label, text) in enumerate(dataloader):
            predited_label = model(text.to(device))
            # predited_label = model(text)
            total_acc += (predited_label.argmax(1) == label.to(device)).sum().item()
            total_count += label.size(0)
            # total_acc += (predited_label.argmax(1) == label).sum().item()
            # total_count += label.size(0)
    return total_acc/total_count

import torch.optim as optim

# 假设 vocab 和 train_iter 已定义
vocab_size = len(vocab)

embed_dim = 64
hidden_dim = 128
num_class = len(set([label for (label, _) in train_iter]))
```

```python
# 创建模型
fc_model = FullyConnectedModel(vocab_size, embed_dim, num_class).to(device)
cnn_model = CNNModel(vocab_size, embed_dim, num_class).to(device)
rnn_model = RNNModel(vocab_size, embed_dim, hidden_dim, num_class).to(device)
model = cnn_model
num_epochs = 30
learning_rate = 0.001
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(cnn_model.parameters(), lr=learning_rate)
for epoch in range(num_epochs):
    train_acc = train(model, train_dataloader, optimizer, criterion)
    print(f'Epoch: {epoch}, Train accuracy: {train_acc:.4f}')
test_acc = evaluate(model, test_dataloader)
print(f'Test accuracy: {test_acc:.4f}')
```

结论分析与体会：

训练结果如下图

```
F3 D:\大学\大二上\深度学习\实验课\实验4>
Epoch: 0, Train accuracy: 0.4719
Epoch: 1, Train accuracy: 0.3830
Epoch: 2, Train accuracy: 0.5204
Epoch: 3, Train accuracy: 0.5052
Epoch: 4, Train accuracy: 0.5161
Epoch: 5, Train accuracy: 0.4639
Epoch: 6, Train accuracy: 0.5426
Epoch: 7, Train accuracy: 0.6429
Epoch: 8, Train accuracy: 0.6163
Epoch: 9, Train accuracy: 0.6300
Epoch: 10, Train accuracy: 0.5960
Epoch: 11, Train accuracy: 0.7300
Epoch: 12, Train accuracy: 0.7128
Epoch: 13, Train accuracy: 0.7097
Epoch: 14, Train accuracy: 0.6396
Epoch: 15, Train accuracy: 0.6882
Epoch: 16, Train accuracy: 0.7579
Epoch: 17, Train accuracy: 0.7979
Epoch: 18, Train accuracy: 0.8241
Epoch: 19, Train accuracy: 0.8229
Epoch: 20, Train accuracy: 0.7755
Epoch: 21, Train accuracy: 0.7755
Epoch: 22, Train accuracy: 0.7449
Epoch: 23, Train accuracy: 0.7684
Epoch: 24, Train accuracy: 0.7368
Epoch: 25, Train accuracy: 0.7629
Epoch: 26, Train accuracy: 0.7473
Epoch: 27, Train accuracy: 0.7216
Epoch: 28, Train accuracy: 0.7684
Epoch: 29, Train accuracy: 0.7200
```

通过本次实验，成功实现了基于全连接网络、CNN 的文本分类模型，并初步了解了 RNN 和 Transformer 在文本分类任务中的应用。通过调整学习率、添加正则化项、等方法进行了模型优化。
就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1－3 道问答题：训练过程中一切正常，训练错误率逐渐减低，准确率逐渐升高，但是一到测试集准确率会降低到 0.21 左右，不清楚为什么。